Pitch and Overview

What we built, why it matters, and how it meets every requirement

CoolRoute: Pedestrian Routing with Shade and Heat-Exposure Budgets

One line: CoolRoute is a walking route planner that finds the shortest route and a cooler route, then explains in plain words what the cooler route costs and gives.

1. The problem

The shortest walking route is often the hottest. It may run along open roads with no shade, cross wide junctions and make people stand in the sun at a red light. Normal map apps count distance and time. They do not count heat.

In a hot city like Jaipur this matters most for older people, children, workers who walk to work, tourists and people who cannot use steps. A good planner should (1) count shade, time of day, effort and accessibility, and (2) say honestly when the cooler route is too long or when its data is weak.

2. Our solution

- Streets and crossings are a **graph**. Each street piece has length, shade, slope, step-free flag, crossing wait and a data-confidence value.
- We compute two routes: the **shortest route** and the **CoolRoute**, which is the shortest route that still fits a **heat budget** the user sets.
- **Step-free** is a hard rule. We never trade it away for shade.
- The app shows a **plain-words trade-off** ('23 m longer, 40% less heat'), flags **missing data**, and never presents heat as a medical risk.

Synthetic neighbourhood at 1 pm: two route choices

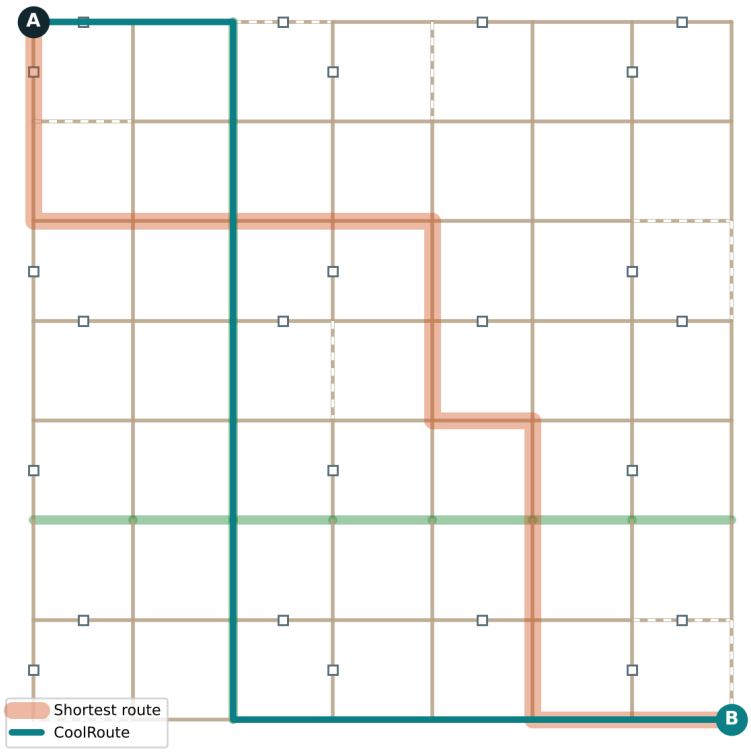


Figure 1. Shortest route (orange halo) and CoolRoute (teal) between A and B at 1 pm. Green streets are tree-lined.

3. Result on the demo trip

Measure	Shortest route	CoolRoute	Change
Distance	1107 m	1129 m	+23 m (+2.1%)
Walk time (with crossing waits)	15.0 min	14.6 min	-0.4 min
Heat exposure	762	455	-40%
Shaded share of the walk	17%	49%	+32 points

Over 148 random long trips at 1 pm with a 60% budget, the median route cut heat by **40%** for a median **3.2%** extra distance (details in file 07).

4. Does it cover every requirement?

Requirement from the problem statement	How we cover it	Where to see it
Map or synthetic neighbourhood with at least two route choices	8 x 8 street grid, two routes always drawn	Prototype map
Baseline shortest-path route and an exposure-aware route with trade-offs	Dijkstra baseline + Lagrangian heat-budget route; plain-words verdict and comparison table	Prototype: 'Which route should I walk?'
Scenario controls: time of day, shade, closed footpath	Time slider, shade toggle, tap-to-close footpath	Prototype: Scenario controls
Graph with length, shade, slope/accessibility, time-dependent heat	All five attributes on every street piece; heat uses a solar model	File 06 and 04
Optimise a heat budget with hard constraints such as step-free	Budget slider; step-free removes non-step-free segments before searching	Prototype toggle; test 6 and 7
Show missing data; no precise personal medical risk	Dashed streets, warnings, data-confidence row; note on every result	Prototype map and warnings

5. What is real and what is a plan

> **Real and tested:** the routing engine, the heat model, the browser prototype, a working JSON backend, and 25 automated checks. **Plan for real cities:** loading OpenStreetMap streets, tree-cover and building-shadow data for Jaipur, and training the shade model on that data. The prototype uses a synthetic neighbourhood, and the ML demo uses synthetic data, so its accuracy numbers show the pipeline works. They do not prove real-world accuracy.

6. Why this can win

- It is **complete**: map, two routes, explanation, scenarios, hard constraints, missing-data warnings.
- It is **honest**: it shows when the cooler route is too long or the data is weak, which is what the problem statement asks for.
- It is **proven**: tests for failure cases, and the same results in Python and JavaScript on 60 random scenarios.
- It is **simple to explain**: one formula for heat, one knob for the budget, one sentence for the trade-off.
- It has a **business path**: city licences, a route API, and a premium app (file 08).

7. The package

File	What it contains
01 Pitch and Overview	This document
02 App Flow	User journey, system flow, edge cases
03 Frontend	Screens, stack, design rules, code structure
04 Backend and Architecture	Services, data model, algorithm, deployment
05 API and Data Sources	Our API, example calls, outside data sources
06 Maths and ML	All formulas with worked examples and charts
07 Test Evidence	Failure-case tests and experiment results
08 Revenue and Business Model	Customers, pricing, 3-year plan, risks
09 Pitch Script and Q&A;	5-minute script, demo steps, judge questions
10 Roadmap and Team Plan	36-hour plan, next steps, setup